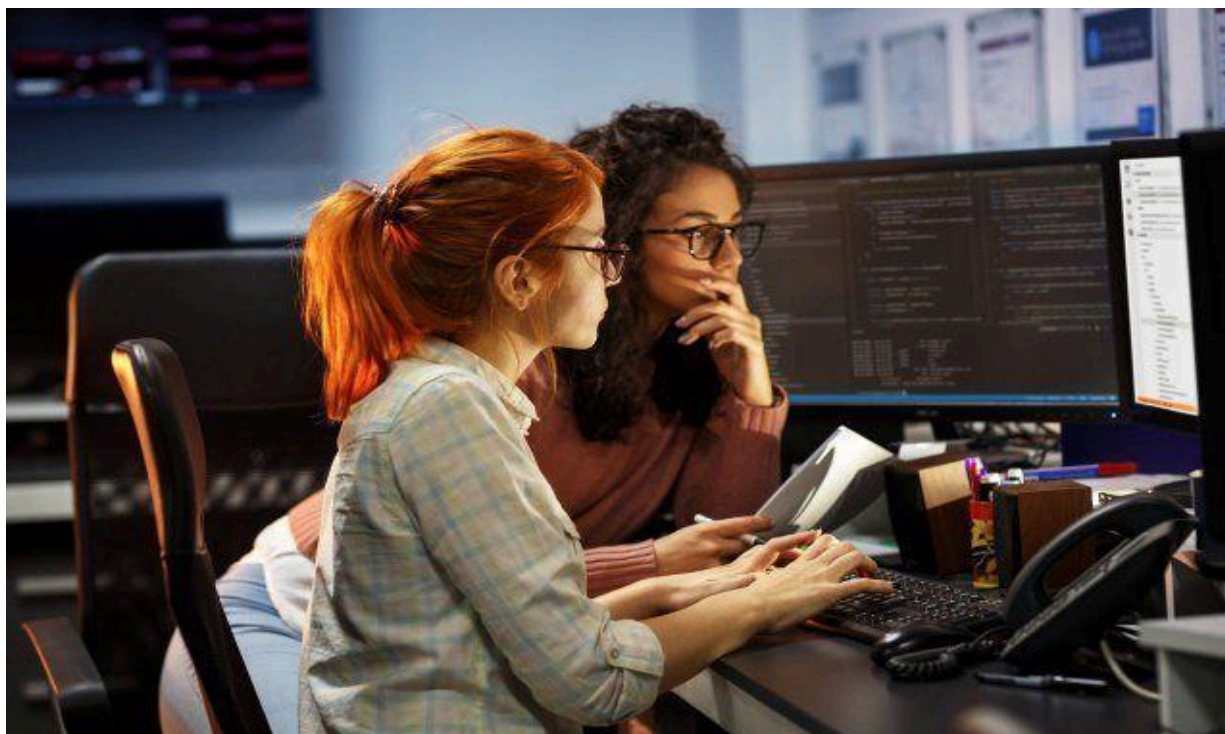




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 2

SEMANA 6

Tema: Programación y Principios de Diseño Orientado a Objetos.

Autor:

Introducción:

Bienvenidos al material de lectura #6 de la Unidad #2 de la materia Programación Orientada a Objetos. En este documento exploramos los conceptos de herencia, uso de super y constructores, sobrescritura con `@Override`, los métodos fundamentales de la clase `Object`, así como el uso de atributos y métodos `protected` y clases y miembros final.

Este material ha sido diseñado para brindarte una base sólida en el ámbito del desarrollo de software, preparándote para afrontar los retos y oportunidades que surgirán en tu trayectoria profesional. A medida que avances en el contenido, te animamos a involucrarte activamente en los ejemplos, ventajas, limitaciones y ejercicios propuestos, ya que contribuirán significativamente a tu proceso de aprendizaje.

[illegible]

- Cuando la jerarquía fuerza dependencias innecesarias.
- Cuando limita la flexibilidad (una clase solo puede extender de una clase padre en Java).

Analogía del mundo real:

Piensa en una familia: un hijo hereda rasgos de sus padres (ojos, cabello), pero puede tener sus propias características únicas. En Java, una clase hija hereda atributos y métodos del padre, pero también puede tener los suyos propios.

Sintaxis básica:

```
class Hija extends Padre { /* ... */ }
```

2.1.1 Clases padre e hija

En herencia, la **clase padre** define atributos y comportamientos comunes, mientras que la **clase hija** hereda estas características y puede agregar las suyas o especializar comportamientos.

Se utiliza la palabra clave **extends**.

Ventajas

- Reutilización de código (no repetir lo común).
- Diseño más claro y organizado.
- Permite polimorfismo: tratar objetos de la hija como si fueran del padre.

Limitaciones

- Acoplamiento fuerte: cambios en el padre afectan a los hijos.
- No todas las relaciones son herencia (a veces conviene **composición**).
- Java solo admite herencia simple (una clase padre).

Analogía

Persona → Estudiante

Un estudiante tiene todo lo que tiene una persona (nombre, edad), pero además posee características propias (carnet, cursos).

Clase padre

```
class Persona {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    public String getNombre() { return nombre; }  
    public int getEdad() { return edad; }  
}
```

Clase hija

```
// Clase hija  
class Estudiante extends Persona {  
    private String carnet;  
  
    public Estudiante(String nombre, int edad, String carnet) {  
        super(nombre, edad); // Llama al constructor de Persona  
        this.carnet = carnet;  
    }  
  
    public String getCarnet() { return carnet; }  
}
```

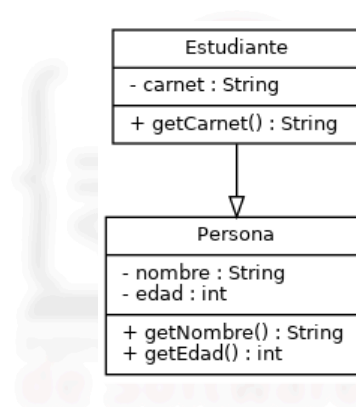
Clase Prueba

```
// Clase de prueba
public class Main {
    public static void main(String[] args) {
        Estudiante e = new Estudiante("Ana", 20, "U2025-001");

        // Uso de métodos heredados
        System.out.println("Nombre: " + e.getNombre());
        System.out.println("Edad: " + e.getEdad());

        // Uso de método propio de Estudiante
        System.out.println("Carnet: " + e.getCarnet());
    }
}
```

Aquí vemos que **Estudiante** es una **Persona**, pero con atributos adicionales.



Persona (padre): contiene **nombre**, **edad** y sus getters.

Estudiante (hija): añade **carnet** y su getter.

La flecha indica herencia (**extends**).

2.1.2 Uso de **super** y ejecución de constructores

¿Para qué existe **super**?

super sirve para:

1. **Invocar el constructor de la clase padre** desde el constructor de la clase hija.

2. **Acceder a métodos o atributos del padre** cuando fueron sobrescritos u ocultos en la hija (p. ej., `super.metodo()`).

Regla clave: la llamada a `super(...)` (o `this(...)`) **debe ser la primera instrucción** del constructor. Solo una de las dos puede estar presente.

Orden real de ejecución al crear un objeto (Java)

Cuando haces `new Hija(...)` ocurre, en este orden:

- Si las clases no estaban cargadas: **se ejecutan los bloques `static` del Padre y luego los del Hija.**
- Se **reserva memoria** y se asignan **valores por defecto.**
- En el constructor de la hija, Java revisa la **primera línea**:
 - Si hay `this(...)`, encadena a otro constructor de la hija.
 - Si hay `super(...)`, sube a construir la parte padre.
 - Si no hay nada, Java **inserta implícitamente `super()`.**
- Antes de ejecutar el **constructor del padre** se hacen:
 - **Inicializadores de instancia** del padre (campos con valores y bloques `{}` no estáticos).
- Se ejecuta el **constructor del padre.**
- Regresa a la hija.
- Antes de ejecutar el **constructor de la hija** se hacen:
 - **Inicializadores de instancia** de la hija.
- Se ejecuta el **constructor de la hija.**
- El **objeto queda listo.**

Esto asegura que **la parte padre queda correctamente inicializada antes** de la parte hija.

Casos típicos y ejemplos

A) El padre **no** tiene constructor vacío: debes llamar **super(args)**

```
class Padre {  
    private final String titulo;  
    public Padre(String titulo) {  
        this.titulo = titulo;  
        System.out.println("Padre listo: " + titulo);  
    }  
}  
  
class Hija extends Padre {  
    public Hija() {  
        // super(); // ERROR: Padre no tiene constructor sin parámetros  
        super("Ingeniera de Software"); // ✅ Obligatorio  
        System.out.println("Hija lista");  
    }  
}
```

B) El padre **sí** tiene constructor vacío: **super()** es implícito

```
class A {  
    public A() {  
        System.out.println("A()");  
    }  
}  
  
class B extends A {  
    public B() {  
        // Aquí Java inserta implícitamente super();  
        System.out.println("B()");  
    }  
}  
  
class Main {  
    public static void main(String[] args) {  
        new B();  
        // Salida:  
        // A()  
        // B()  
    }  
}
```

C) Encadenamiento de constructores con this(...) y super(...)

- **No** puedes poner this(...) y super(...) en la **misma** función constructora.
- Puedes encadenar **entre constructores de la propia clase** y en **uno** de ellos llamar a super(...)

```
class Persona {  
    private final String nombre;  
    private final int edad;  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre; this.edad = edad;  
        System.out.println("Persona(" + nombre + ", " + edad + ")");  
    }  
}  
  
class Estudiante extends Persona {  
    private final String carnet;  
  
    public Estudiante(String nombre) {  
        this(nombre, 18, "SIN-CARNET"); // Encadena a otro ctor de Estudiante  
    }  
  
    public Estudiante(String nombre, int edad, String carnet) {  
        super(nombre, edad); // Subimos al padre  
        this.carnet = carnet; // Ya de regreso en la hija  
        System.out.println("Estudiante(" + carnet + ")");  
    }  
}
```


D) Llamar métodos del padre: super.metodo()

```
class Logger {  
    public void log(String msg) { System.out.println("[BASE] " + msg); }  
}  
  
class LoggerApp extends Logger {  
    @Override  
    public void log(String msg) {  
        super.log(msg); // Reutilizo el comportamiento del padre  
        System.out.println("[APP] " + msg.toUpperCase());  
    }  
}
```

Buenas prácticas con super y constructores

- **Centraliza inicialización:** usa this(...) para evitar duplicar lógica y en **un solo** constructor coloca super(...).
- Mantén los constructores **ligeros**: evita lógica pesada o efectos secundarios; mejor usa fábricas o métodos init.
- **No llames métodos sobrescribibles** desde el constructor del padre***: si la hija los sobrescribe y usan campos aún no inicializados, puedes provocar estados inconsistentes.
- Si el padre define invariantes (campos final, validaciones), **llama al constructor adecuado** con super(...).

Errores frecuentes (y cómo evitarlos)

1. **Falta super(args)** cuando el padre no tiene constructor vacío → **Compila con error.**

2. Intentar usar **a la vez** `this(...)` y `super(...)` en el mismo constructor → **No compila.**
3. Llamar a métodos que dependen de estado aún no inicializado desde el constructor del padre → **comportamiento inesperado.**
4. Olvidar el orden real de inicialización (padre antes que hija) → **bugs sutiles.**

2.1.3 Métodos protegidos y sobrescritos (@Override).

En herencia, una subclase puede **redefinir** (o sobrescribir) un método heredado para cambiar su comportamiento.

Esto se llama **override** (sobrescritura).

- La palabra clave `@Override` es una **anotación** que indica al compilador: *“este método redefine uno de la superclase”*.
- Sirve como **verificación automática**: si te equivocas en la firma, el compilador marcará error.

Diferencia con sobrecarga (overload):

- **Override**: misma firma (nombre y parámetros) → redefine comportamiento.
- **Overload**: mismo nombre, **distintos parámetros** → método adicional, no reemplazo.

Reglas para sobrescribir

1. La firma debe ser **idéntica** (nombre + parámetros).
2. El tipo de retorno debe ser igual o covariante (subtipo permitido).
3. El modificador de acceso debe ser **igual o más permisivo**:
 - Ejemplo: de `protected` a `public` sí.
 - De `protected` a `private` no.

4. No se pueden sobrescribir métodos **final**, **static** o **private**.

Uso de métodos **protected**

- Un método **protected** puede ser sobrescrito por subclases.
- Es accesible desde **subclases**, aunque estén en paquetes diferentes.
- Favorece la extensión de clases cuando se espera que las hijas redefinan el comportamiento.

Ejemplo en código

Clase Padre

```
class Animal {  
    protected void sonido() {  
        System.out.println("(silencio)");  
    }  
}
```

Clase hija sobrescribiendo al metodo

```
class Perro extends Animal {  
    @Override  
    protected void sonido() {  
        System.out.println("Guau!");  
    }  
}
```

Clase principal Main

```
public class Main {  
    public static void main(String[] args) {  
        Animal a = new Animal();  
        Animal b = new Perro();  
  
        a.sonido(); // (silencio)  
        b.sonido(); // Guau!  
    }  
}
```

polimorfismo: aunque **b** se declara como **Animal**, el método sobrescrito en **Perro** se ejecuta.

Ventajas de sobrescribir métodos

- Permite especializar comportamientos heredados.
- Facilita polimorfismo y reutilización de código.
- Hace el diseño más flexible y adaptable a nuevos requerimientos.

Limitaciones y malas prácticas

- Sobrescribir sin necesidad puede volver el código confuso.
- Modificar **equals()** o **hashCode()** sin coherencia rompe estructuras como **HashMap**.
- No usar **@Override** aumenta riesgo de errores sutiles (por ejemplo, cambiar un parámetro accidentalmente).

2.1.4 La clase **Object** y sus métodos (**toString**, **equals**, etc.).

En Java, todas las clases heredan implícitamente de **java.lang.Object**. Incluso si no escribes **extends**, la clase lo hace automáticamente.

Esto significa que cualquier objeto en Java tiene los métodos definidos en **Object**

Métodos más importantes de **Object**

1. **toString()**

- Devuelve una representación textual del objeto.
- Por defecto, imprime: *nombreDeClase@códigoHash*.

- Se suele sobrescribir para mostrar contenido relevante.

```
@Override
public String toString() {
    return "Producto: " + nombre + " (código: " + codigo + ")";
}
```

2.equals(Object o)

- Compara dos objetos para saber si son “lógicamente iguales”.
- Por defecto, se comporta como `==` (compara referencias).
- Se sobrescribe para comparar **contenido** en lugar de direcciones de memoria.

```
@Override
public boolean equals(Object o) {
    if (this == o) return true; // misma referencia
    if (o == null || getClass() != o.getClass()) return false;
    Producto p = (Producto) o;
    return codigo.equals(p.codigo);
}
```

3.hashCode()

- Devuelve un número entero asociado al objeto.
- Si sobrescribes `equals()`, **debes sobrescribir también `hashCode()`**, o habrá problemas en colecciones como `HashMap`, `HashSet`.

```
@Override
public int hashCode() {
    return codigo.hashCode();
}
```

Nota: Investigue que otros métodos hacen falta

Ventajas de sobrescribir estos métodos

- Hace que los objetos sean **legibles** (`toString`).
- Permite **comparación lógica** (`equals`, `hashCode`).
- Es fundamental para trabajar con **colecciones** (`HashMap`, `HashSet`, `List`).

Malas prácticas

- Usar `==` en lugar de `equals()` para comparar contenido.
- Sobrescribir `equals()` sin sobrescribir `hashCode()`.
- No considerar `null` en `equals()` → `NullPointerException`.
- Usar `finalize()` como mecanismo de liberación de recursos (ya no recomendado).

2.2 Métodos y atributos protegidos.

En Programación Orientada a Objetos (POO), la visibilidad o nivel de acceso determina qué tan accesibles son los atributos (variables) y métodos (funciones) de una clase desde otras clases.

Además de `public` y `private`, existe un tercer modificador muy importante: **`protected` (protegido)**.

Atributos protegidos: Son variables de una clase que no son accesibles directamente desde cualquier clase externa, pero sí pueden ser heredadas y utilizadas en las subclases (clases hijas).

Métodos protegidos: Son funciones que siguen la misma lógica: pueden ser usadas por la propia clase y sus subclases, pero no están disponibles de forma pública para cualquier otra clase.

En Java, se declaran con la palabra clave **`protected`**.

```
protected int edad;
protected void mostrarEdad() {
    System.out.println("La edad es: " + edad);
}
```

Accesibilidad comparada

Modificador	Misma clase	Mismo paquete	Subclases (otro paquete)	Otras clases externas
private	Permitido	No permitido	No permitido	No permitido
default	Permitido	Permitido	No permitido	No permitido
protected	Permitido	Permitido	Permitido	No permitido
public	Permitido	Permitido	Permitido	Permitido

protected es más restrictivo que **public**, pero más abierto que **private**.

Clase Padre

```
class Animal {
    protected String nombre;

    protected void comer() {
        System.out.println(nombre + " está comiendo...");
    }
}
```

Clase hija

```
class Perro extends Animal {  
  
    public void mostrarInfo() {  
        // Accedemos al atributo y método protegidos  
        nombre = "Firulais";  
        comer();  
    }  
}
```

Clase Externa

```
public class Main {  
    public static void main(String[] args) {  
        Perro p = new Perro();  
        p.mostrarInfo();  
  
        // ✗ Error: no se puede acceder directamente a "nombre" ni a "comer()"  
        // p.nombre = "Boby";  
        // p.comer();  
    }  
}
```

Ventajas de **protected**

1. **Facilita la herencia:** permite que las clases hijas reutilicen atributos y métodos del padre sin ser públicos.
2. **Mayor encapsulamiento:** restringe el acceso para proteger datos sensibles frente a clases externas.
3. **Reutilización de código:** evita duplicación en subclases.

Desventajas de **protected**

1. **Puede romper el encapsulamiento:** si se abusa, las subclases pueden modificar atributos internos de manera no controlada.
2. **Dificulta el mantenimiento:** muchas clases hijas pueden depender de variables protegidas, haciendo difícil cambiar la implementación.
3. **Exposición innecesaria:** en algunos casos sería mejor usar **private** con métodos **get/set** en lugar de **protected**.

2.3. Clases Finales en Java.

Qué es una clase final?

En Java, una **clase final** es una clase que **no puede ser heredada**. Se declara con la palabra reservada **final** antes de la palabra **class**.

```
public final class Constantes {  
    public static final double PI = 3.1416;  
    public static final String UNIVERSIDAD = "UES";  
}
```

Esto significa que ninguna otra clase podrá extenderla con **extends**.

Por qué existen las clases finales?

1. **Seguridad y consistencia:**
Si un comportamiento debe mantenerse sin alteraciones, la clase se marca como **final**. Ejemplo: la clase **String** en Java es final para evitar que alguien cambie su funcionamiento básico.
2. **Optimización del compilador:**
El compilador y la JVM pueden hacer ciertas optimizaciones sabiendo que la clase no será heredada.

3. Diseño intencional:

Permite expresar de forma clara que esa clase **no está pensada para extenderse**, sino para usarse tal como está.

```
// Clase final
public final class CuentaBancaria {
    private double saldo;

    public CuentaBancaria(double saldoInicial) {
        this.saldo = saldoInicial;
    }

    public void depositar(double monto) {
        saldo += monto;
    }

    public void retirar(double monto) {
        if (saldo >= monto) {
            saldo -= monto;
        } else {
            System.out.println("Fondos insuficientes.");
        }
    }

    public double getSaldo() {
        return saldo;
    }
}

// Intento de herencia (no permitido)
public class CuentaPremium extends CuentaBancaria { // X Error de compilación
    public CuentaPremium(double saldoInicial) {
        super(saldoInicial);
    }
}
```

La clase **CuentaBancaria** al ser **final**, no puede ser extendida, lo que protege su lógica interna.

Diferencia con métodos y variables **final**

- Clase final: No se puede heredar.
- Método final: No puede ser sobrescrito (**@Override**).
- Variable final: Su valor no puede cambiar (constante).

Ejemplo

```
public class Figura {  
    public final void dibujar() {  
        System.out.println("Dibujando figura...");  
    }  
}
```

Ventajas

- Garantiza **inmutabilidad** en ciertos objetos (ej: **String**).
Mejora la **seguridad** evitando que se modifique la lógica base.
- Claridad en el diseño: el programador indica que no debe extenderse.

Desventajas

- Limita la **reutilización** de código mediante herencia.
- Puede obligar a usar **composición** en lugar de herencia.

